

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
A Constituent Unit of MAHE, Manipal

ADVANCED TECHNOLOGY LAB (ICT- 3241)

WardWatch: A Flutter and Supabase Integrated Mobile Application for Civic Issue Reporting and Municipal Analytics

by

Name: Asmita Guha
Reg No.: 230911112
Section: IT-D
Roll No.: 19

B. Tech Information Technology

Semester- VI

Under the guidance of

Dr. Ashwath Rao B
Assistant Professor - Selection Grade
School of Computer Engineering
Manipal Institute of Technology
ashwath.rao@manipal.edu

Academic Year: 2025-2026

Date: April 22, 2026

Contents

1 Title **2**

2 Introduction to the App **2**

3 Literature Review **2**

4 Methodology **3**

4.1 Technology Stack & Integration 3

4.2 System Architecture & Database Schema 4

4.3 System Diagrams 5

4.3.1 Use Case Diagram 5

4.3.2 High-Level Database Architecture (ER Diagram) 5

4.3.3 System Class Diagram 6

4.4 Testing Strategy 6

5 Results **6**

6 Conclusion **8**

6.1 Achievements 8

6.2 Limitations 8

6.3 Future Work 9

I. Title

WardWatch: A Flutter and Supabase Integrated Mobile Application for Civic Issue Reporting and Municipal Analytics.

II. Introduction to the App

Municipal authorities have to deal with the problem of keeping city infrastructure in good shape over and over again. Potholes, broken streetlights, water leaks, overflowing trash cans, and other damaged property are some of the problems with municipal infrastructure that often go unrepaired because it is hard for citizens to communicate with municipal representatives. Older ways of communicating, like helplines or generic email portals, don't give citizens clear information about where they are, how long it will take to get there, or how to track their progress in real time. This often leads to unresolved issues coupled with frustratingly slow response times.

WardWatch is a mobile application designed to address this issue. It connects residents with their local governments online. Residents can report different kinds of local problems using photos and accurate GPS coordinates through a clear, localised, and easy-to-reach system. In addition, the application allows municipal administrators to have a centralized "Command Center" with real-time analytics and a master queue for efficient use of resources and measurable resolution of issues.

III. Literature Review

In the last ten years, the use of civic-tech apps and e-governance has grown rapidly. The growth of civic-tech apps and e-governance is a change from simple web portals to smart city ecosystems that work together. Early proprietary applications such as *SeeClickFix* [1] and localised community forums like *Nextdoor* [2] established the foundation for digital civic engagement by enabling citizens to publicly express grievances. A lot of the time, these apps work as separate, third-party ticketing systems that do not use deep, real-time integration with the city's internal databases.

Recent literature underscores the pressing need for direct mobile crowdsensing to scale urban data collection. Wang et al. [3] proposed the use of reinforcement learning to perform a mobile crowdsensing study to understand what parts of an urban area have infrastructure needs and then create a planning map of those needs by using everyday citizens' cellular devices. This approach provides a way to gather certain types of data without incurring the high cost associated with deploying a traditional fixed-sensor network. Building upon this, Binu et al. [4] presented a methodology for integrating the use of cellular telephones' global positioning systems (GPS) to log citizen complaints. Their research indicated that combining GPS data with photographs of the complaints resulted in a large decrease in the administrative work required to fulfill the requests, giving citizens greater power and reducing the amount of false reporting made to public

agencies.

To address the challenges associated with managing this influx of data, Aazam et al. [5] suggested that smart cities establish a cloud-based architecture rather than using outdated legacy systems to capture and process citizen-generated data so that they can be provided with real-time responses to citizen concerns regarding the optimised collection and management of waste and assets. For administrative oversight, Han et al. [6] evaluated the design and implementation of Smart City Dashboards for the management of the numerous and varied types of data generated by citizens in a smart city. They highlighted how visual analytics allow cities to centralise their visualisation of the data citizens generate, and this enables municipal authorities to provide better response times to citizen-generated data. This also helps municipalities improve the efficiency of how they allocate their resources. Finally, Bommu et al. [7] emphasised the need for accurate routing analysis to be performed at the network level, as well as the establishment of immutable audit trails for citizen-generated data and the need to ensure that both IoT-generated data and citizen-generated data are secure from unauthorised alteration.

WardWatch addresses the limitations identified in existing literature by going beyond a simple web portal and using a standalone platform that is fully integrated (such as *SeeClickFix* [1]). Utilizing a data streaming pipeline to create real-time updates to administrative dashboards effectively address the latency issues identified by Aazam et al. [5]. Additionally, the use of dual write cryptographic audit trail and strict RBAC (Role-Based Access Control) creates separate user access rights for Resident Portal and Municipal Authority Access in one secure mobile application environment.

IV. Methodology

WardWatch was developed using an agile methodology and took full advantage of a modern, cross-platform technology stack to deliver high performance, security, and rapid deployment across multiple mobile operating systems.

A. Technology Stack & Integration

- **Frontend (Flutter):** Built using the Flutter SDK (Dart) [8], WardWatch can be natively compiled for both iOS and Android platforms from a single code base. It utilizes a declarative user interface architecture that allows for advanced visual implementations, such as glassmorphism and custom squircle icons. The app also utilizes localized device hardware interfaces to capture high-fidelity media through the use of `image_picker` and capture longitude and latitude with `geolocator`.
- **Backend (Supabase & PostgreSQL):** Supabase [9] is the Backend-as-a-Service (BaaS) that the system uses. PostgREST [10] automatically exposes the core database, which is a strong, relational PostgreSQL instance, as a RESTful API.
- **Real-Time Pipelines:** WardWatch uses Supabase's real-time WebSockets to enable real-time database changes. This means that citizens and authorities no longer

have to wait for manual updates – they can instantly synchronise data across the platform. This translates to the Admin Command Center queue being updated in real time, even when a resident submits a new report.

B. System Architecture & Database Schema

The application uses a star-schema approach that centres around the `reports` table.

- **RBAC & Row Level Security (RLS):** Supabase Auth (JWTs) is a safe way to authenticate users. These JSON Web Tokens work with PostgreSQL's RLS policies to allow residents to only read/write their own tickets. Authenticated municipal administrators can have more permissions while logged in.
- **Cascading Deletions:** The main data hub (`reports`) is connected to satellite tables like `locations`, `media`, and `status_logs` via foreign keys. All of these keys have the setting `ON DELETE CASCADE`, which means that the database will automatically clean up anything that gets deleted, without needing the admin to manually remove all of the associated data. For example, if an admin deletes a parent report, all of the associated GPS and photo data gets deleted automatically.
- **Resolution Engine & Auditing:** To help administrators keep track of their work and do research, a dual-write system was employed. A database transaction updates the status of a report from the `reports` table when a report is resolved and also simultaneously writes a permanent log to the `status_logs` table with a timestamp. This log is cryptographically tied to the `auth.users.id` of the resolving official from the auth.users table and cannot be changed by an unauthorised user.

C. System Diagrams

4.3.1 Use Case Diagram

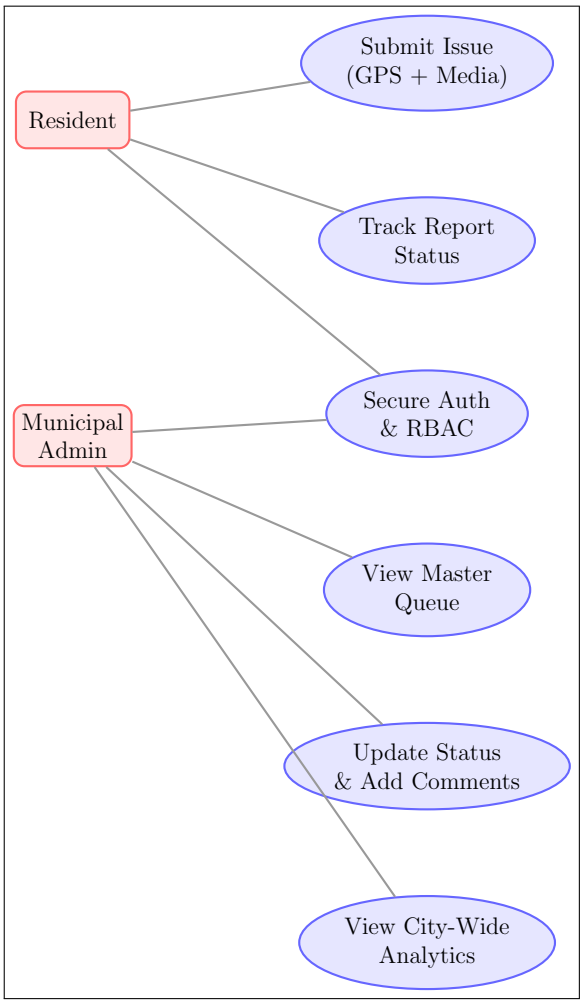


Figure 1: WardWatch Use Case Diagram: Dual-role functionalities

4.3.2 High-Level Database Architecture (ER Diagram)

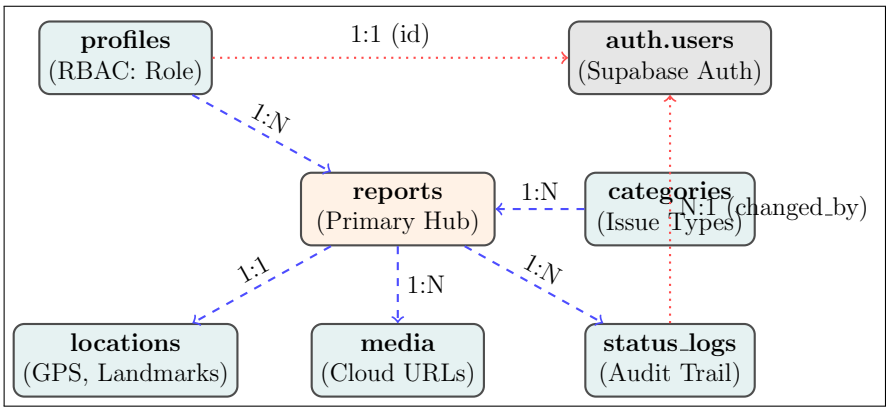


Figure 2: Entity-Relationship Diagram: Relational structure and Auth association

4.3.3 System Class Diagram

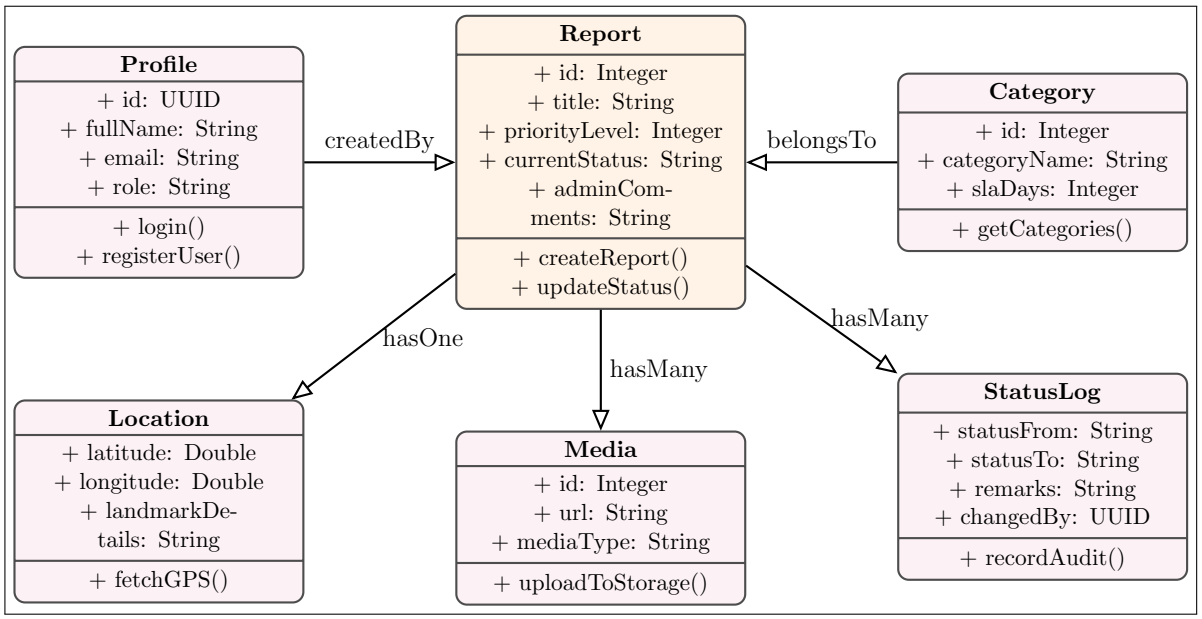


Figure 3: UML Class Diagram: Database Schema Properties

D. Testing Strategy

- **Unit Testing:** Unit testing for all of the different authentication methods was put in place to ensure that people without access to the dashboard cannot access it. This testing occurred within a matter of milliseconds. In addition, unit testing was done to verify that proper completion of forms occurs before submitting the form.
- **Integration Testing:** The real-time StreamBuilder pipelines have been tested to verify that the Admin Dashboard updates instantly when a resident reports an incident on a different device without requiring a page refresh.
- **Database Constraint Testing:** The Supabase SQL editor functionality was tested to verify that the `ON DELETE CASCADE` constraint works as expected. The Unit Testing and Integration Testing code was written and tested with the functionality, ensuring that all child tickets, child photo records and child status records are properly deleted when a parent ticket created by that admin is deleted.

V. Results

The application successfully met all initial design parameters. The intuitive UI makes it easy to report quickly, and the backend handles the real-time queue quickly. The Application Screenshots are as follows:



Figure 4: Landing Page

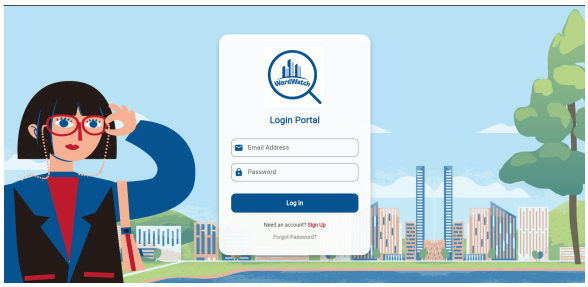


Figure 5: Login

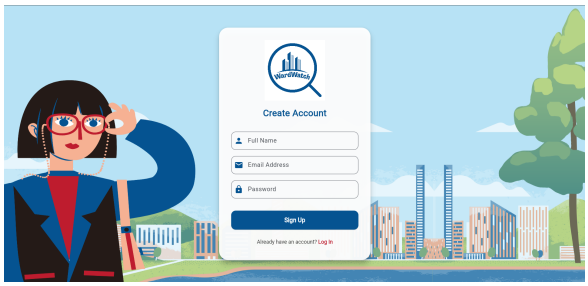


Figure 6: Sign Up

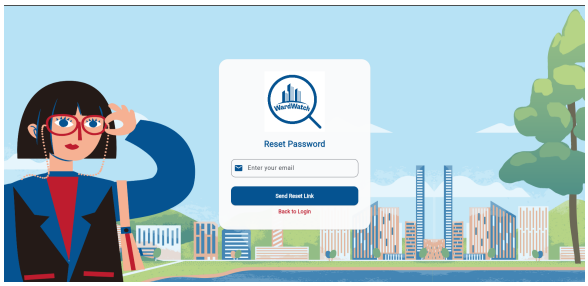


Figure 7: Reset Password

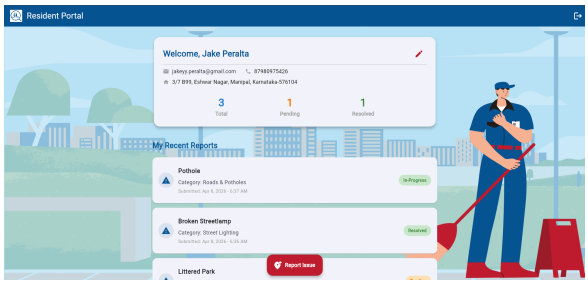


Figure 8: Resident Dashboard

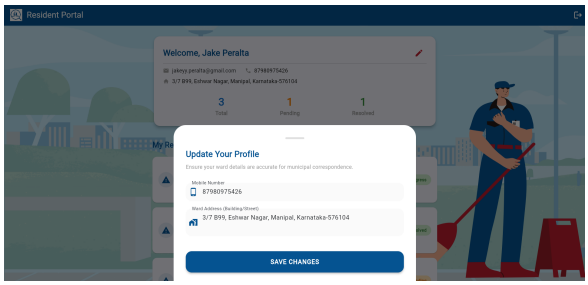


Figure 9: Update Resident Profile

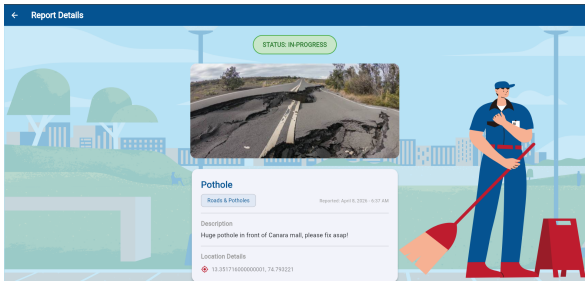


Figure 10: Report Details

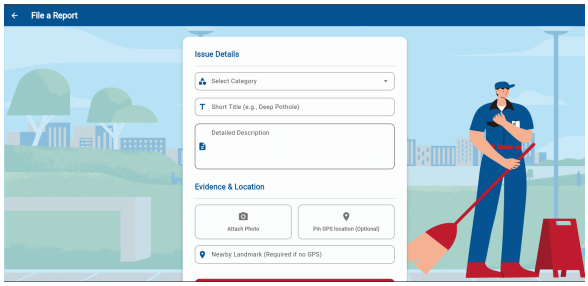


Figure 11: File a New Report

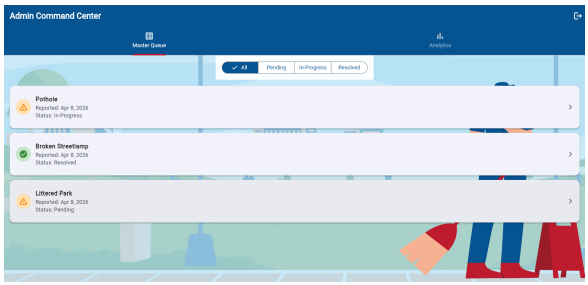


Figure 12: Admin Master Queue

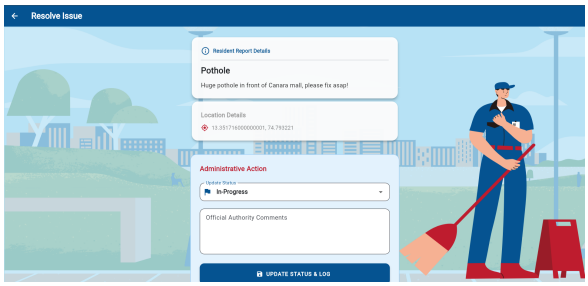


Figure 13: Admin Issue Handler

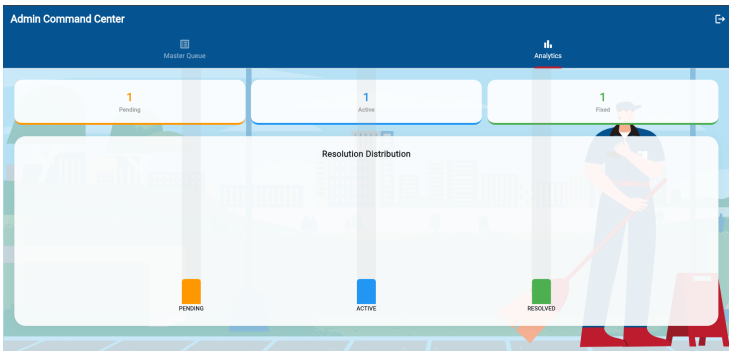


Figure 14: Area-wide Analytics

VI. Conclusion

A. Achievements

It exemplified how well a dual system combining Flutter's reactive UI with Supabase's real-time PostgreSQL backend will work together to create a real-time tracking application with auditing for municipalities. It also illustrated how effective the dual-write auditing system (`status_logs`) was in creating a verifiable and proper chain of accountability for municipal employees and how well the Resident Portal simplified the reporting process.

B. Limitations

- Currently, the need for an active and stable internet connection for pushing media payloads and receiving real-time stream data is a necessary requirement. Offline-first cache, such as that from SQLite or Hive, has not been implemented in low-connectivity areas to date.
- Manual onboarding procedures for administrators are extremely important to ensure that there are no unauthorized users elevated to Admin privileges from the public portal.

C. Future Work

Future iterations of WardWatch could leverage advanced Artificial Intelligence, Machine Learning, and Data Science paradigms. Specifically:

- **AI Duplicate Detection:** An ML reasoned model can be developed to scan incoming report photos to identify and group duplicates of potholes or incidences to help mitigate the occurrence of the municipal queue being flooded with duplicated pothole reports.
- **Automated Routing:** Natural language processing can be applied to help sort and send reports to the department in charge of the reported problem. For example, reports related to power outages can be relayed to the Electricity Department for prompt action.
- **Push Notifications:** Firebase Cloud Messaging (FCM) can be employed to alert residents as soon as an issue reported by them is updated or resolved by the administrator.

References

[1] SeeClickFix, *SeeClickFix Website*, 2026. [Online]. Available: `https://seeclickfix.com/`

[2] Nextdoor, *Nextdoor Website*, 2026. [Online]. Available: `https://nextdoor.com/`

[3] C. Wang et al., "Smart Mobile Crowdsensing With Urban Vehicles: A Deep Reinforcement Learning Perspective," *IEEE Access*, Mar. 2019. [Online]. Available: `https://ieeexplore.ieee.org/document/8667822`

[4] K. Binu, V. K, M. R, A. V. Pillai, and S. Thampi, "SmartReporter: A Crowdsourced Complaint Resolution System," *International Journal of Engineering Research & Technology (IJERT)*, vol. 15, issue no. 1, 2026. [Online]. Available: `https://www.ijert.org/smartreporter-a-crowdsourced-complaint-resolution-system-ijertv15is010067`

[5] M. Aazam, M. St-Hilaire, C. H. Lung, and I. Lambadaris, "Cloud-Based Smart Waste Management for Smart Cities," *2016 IEEE 21st International Workshop on Computer Aided Modelling and Design of Communication Links and Networks (CAMAD)*, Toronto, ON, Canada, 2016. [Online]. Available: `https://ieeexplore.ieee.org/document/7790356`

[6] Q. Han, P. Nesi, G. Pantaleo, and I. Paoli, "Smart City Dashboards: Design, Development, and Evaluation," *2020 IEEE International Conference on Human-Machine Systems (ICHMS)*, Sep. 2020. [Online]. Available: `https://ieeexplore.ieee.org/document/9209493`

[7] S. Bommu et al., "Smart City IoT System Network Level Routing Analysis and Blockchain Security Based Implementation," *Journal of Electrical Engineering & Technology, Springer*, Oct. 2022. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC9549033/

[8] Google, "Flutter: Build apps for any screen," *Flutter Documentation*, 2026. [Online]. Available: https://flutter.dev/docs

[9] Supabase Inc., "The Open Source Firebase Alternative," *Supabase Architecture Docs*, 2026. [Online]. Available: https://supabase.com/docs

[10] PostgREST, "REST API for any Postgres database," *PostgREST Official Documentation*, 2026. [Online]. Available: https://postgrest.org/en/stable/